Reflexion zur Einsendeaufgabe 2

Front-End Development (SED502) · Projekt „Bundesliga-Analyst“ · Thomas Kluczny, 52108080 · Einzelarbeit
Repository: https://github.com/tkluczny/bundesliga-analyst

Ausgangslage und Vorgehen

Ziel war eine React-Anwendung mit TypeScript, die eine öffentliche REST-API und ein sinnvolles KI-Feature verbindet. Ich habe mich für eine Statistik-App zur 1. Fußball-Bundesliga entschieden, weil mich das Thema interessiert und die OpenLigaDB echte, laufend aktualisierte Daten ohne API-Key liefert. Die KI sollte keine isolierte Chatbox sein, sondern die berechneten Zahlen eines Teams oder eines Duells in verständlichen Text übersetzen.

Die Aufgabe ist als Gruppenarbeit gedacht. Ich habe sie bewusst allein umgesetzt, weil ich jeden Teil – API-Anbindung, State-Management, Streaming, Git und Dokumentation – selbst durchlaufen wollte. Den Großteil des Quellcodes habe ich mit Unterstützung von Claude (Anthropic) erstellt. Meine Rolle lag vor allem in den Entscheidungen (Thema, Umfang, KI-Anbieter), im Einrichten der Umgebung, im Testen mit echten Daten und im Eingrenzen von Fehlern. Das schreibe ich hier offen, weil es für die Einordnung meines Lernfortschritts wichtig ist.

Fehler und Stolpersteine

- **Falscher Projektordner.** Beim Entpacken entstand ein verschachtelter Ordner, und ich habe in VS Code zuerst die äußere Ebene geöffnet. Dadurch fanden `npm` und Git die Projektdateien nicht. Seitdem achte ich darauf, dass `package.json` direkt im geöffneten Ordner liegt.
- **Wechsel des KI-Anbieters mitten im Projekt.** Geplant war OpenAI, weil ich dort noch Guthaben hatte. Später habe ich mich für Google Gemini entschieden. Der Wechsel war nur deshalb einfach, weil der API-Aufruf in einer einzigen Datei (`aiStream.ts`) gekapselt war. Ich hätte die Entscheidung trotzdem früher treffen sollen.
- **API-Key auf einem Screenshot.** Beim Nachfragen zu einem Problem habe ich einen Screenshot geteilt, auf dem die geöffnete `.env` mit dem vollständigen Key zu sehen war. Zusätzlich hatte der Copilot-Chat in VS Code die `.env` als Kontext angehängt. Der Key lag zwar nie im Repository, war aber trotzdem nicht mehr geheim. Ich habe ihn daraufhin gelöscht und neu erstellt. Das war mein ernstester Fehler, weil die `.gitignore` allein offensichtlich nicht reicht – Sicherheit hängt auch am eigenen Arbeitsverhalten.
- **KI-Analyse lud scheinbar endlos.** Tabelle und Teamseiten funktionierten, aber bei der Formanalyse hüpften die Ladepunkte minutenlang. Zuerst vermutete ich einen Fehler im Code. Die Messung zeigte zwei andere Ursachen: Das Gemini-Modell „dachte“ standardmäßig über 20 Sekunden nach, selbst bei einer Ein-Wort-Antwort, und zeitweise lehnte Google das Modell wegen Überlastung ab (HTTP 503). Gelöst wurde das durch eine niedrige Denkstufe und automatische Ausweichmodelle; jetzt erscheint der erste Text nach etwa sechs Sekunden. Ehrlich gesagt fehlt aber weiterhin ein Zeitlimit: Hängt die API komplett, wartet die App, statt nach einer gewissen Zeit eine Fehlermeldung zu zeigen.
- **Tippfehler beim GitHub-Repository.** Der erste Push scheiterte mit „Repository not found“, weil ich den Namen auf GitHub falsch geschrieben hatte. Durch Umbenennen in den Repository-Einstellungen habe ich verstanden, dass die Remote-URL exakt zum Namen passen muss.
- **Kleinere Punkte.** Die Git-Ausgabe blieb in einem Pager hängen (Beenden mit `q`), und die Warnungen „LF will be replaced by CRLF“ haben mich zuerst verunsichert, sind aber

nur ein Unterschied der Zeilenenden zwischen Windows und Linux.

Erkenntnisse

- **Externe Dienste sind der unsicherste Teil einer Anwendung.** Der eigene Code kann korrekt sein und die App trotzdem „kaputt“ wirken. Verständliche Fehlermeldungen, Ausweichlösungen und Messen statt Raten sind deshalb genauso wichtig wie die Funktion selbst.
- **Kapselung zahlt sich aus.** Weil API-Zugriff, Streaming und Statistik in eigene Dateien und Custom Hooks ausgelagert sind, betraf jede Änderung nur eine Stelle. Das wird auch für ESA 3 wichtig, wenn der KI-Aufruf hinter eine serverlose Funktion wandert.
- **KI-Texte brauchen Kontrolle.** Ich habe die Analysen stichprobenartig mit den angezeigten Zahlen verglichen (Punkte aus Siegen und Remis, Reihenfolge der Formkurve, Summe der direkten Duelle). Erfundene Fakten sind mir nicht aufgefallen, was ich auf den strikten System-Prompt zurückführe – trotzdem bleibt die Kennzeichnung „KI-generiert“ notwendig.
- **KI-Unterstützung beim Programmieren spart viel Zeit, verschiebt aber die Arbeit.** Statt Code zu tippen, musste ich ihn verstehen, testen und Fehler eingrenzen. Hier sehe ich meine größte Lücke: Die Grundidee von `useFetch`, `useAIStream` und dem Streaming kann ich erklären, die Details (z. B. den Zeilenpuffer beim Lesen der Server-Sent Events) würde ich ohne Vorlage noch nicht selbst schreiben.

Lernfortschritt

Im Vergleich zu ESA 1, einer statischen Seite mit HTML und CSS, bei der ich Git erstmals verwendet habe, war ESA 2 ein deutlicher Sprung. Ich kann jetzt ein Vite-Projekt aufsetzen, verstehe den Aufbau aus Komponenten, Props mit Interfaces und Seiten mit React Router, und weiß, wozu `useState`, `useEffect` mit Abhängigkeits-Array und Custom Hooks dienen. Die drei Zustände Laden, Erfolg und Fehler sind für mich jetzt ein fester Bestandteil jeder Datenabfrage. Neu waren außerdem Umgebungsvariablen, der Umgang mit API-Keys, Streaming von KI-Antworten und das Arbeiten mit einem Remote-Repository (`remote`, `push`, Umbenennen). Bei Git fühle ich mich inzwischen sicher, bei TypeScript-Typen und asynchronem Code noch nicht.

Offene Punkte und Ausblick auf ESA 3

Bewusst offen sind: ein Zeitlimit für KI-Anfragen, automatisierte Tests und der Schutz des API-Keys im Deployment, da Vite-Variablen im Browser-Bundle landen. In ESA 3 möchte ich den KI-Aufruf über eine serverlose Funktion absichern, die App veröffentlichen und die bereits angelegte Barrierefreiheit überprüfen. Vorher nehme ich mir die drei zentralen Dateien (`useFetch.ts`, `useAIStream.ts`, `aiStream.ts`) noch einmal Zeile für Zeile vor, damit ich die Erweiterungen selbstständiger umsetzen kann.